

Sistema Reserva Servicio Comunitario

Este proyecto es una aplicación web desarrollada en Python utilizando el framework FastAPI. Con ella podemos gestionar la reserva de instalaciones comunes (Gimnasio, Piscina, Sauna) en un complejo residencial, garantizando el cumplimiento de aforos limitados.

El sistema implementa una arquitectura asíncrona para manejar múltiples usuarios simultáneos sin bloqueos, autenticación basada en roles (RBAC) y persistencia de datos en memoria.

Tecnologías

- Lenguaje: Python 3.x
- Framework Web: FastAPI
- Servidor: Uvicorn
- Motor de Plantillas: Jinja2
- Concurrencia: asyncio (Python Native Async Library)

Componentes

1. Capa de Datos (app/database)

-models.py:

Contiene dos clases User y service. User es la representación simple de la entidad usuario (id, username, password).

Service, define las propiedades de los servicios, además del uso del Schedule como diccionario anidado optimizado para búsquedas rápidas: { "YYYY-MM-DD": { "HH:MM": [user1, user2] } }.

-store.py:

Actúa como un Singleton que mantiene el estado de la aplicación en la memoria RAM. Inicializa los objetos Service con sus capacidades máximas. Provee métodos helper como get_reservations_for_user que transforman la estructura de datos interna en listas planas para la vista.

2. Capa de Controladores (app/routers)

-auth.py:

Maneja el ciclo de vida de la sesión. Role-Based Access Control, asigna roles en la cookie de sesión firmada. Depende de como accedas a la aplicación.

role="resident": Acceso total || role="guest": Acceso de solo lectura (Read-Only)

-services.py:

Controla la interacción principal del usuario con el sistema.

/dashboard: Orquestador que recupera servicios y reservas. Filtra datos basándose en si el role es "resident" o "guest".

/reserve (POST): Recibe el payload del formulario, invoca el método add_reservation del modelo y maneja el flujo de redirección (Post/Redirect/Get pattern) según el resultado (éxito o error de capacidad).

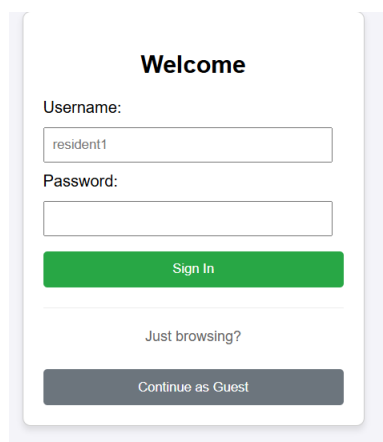
/cancel (POST): Procesa la solicitud de baja de una reserva.

3. main.py

Punto de entrada (Entry Point) de la aplicación ASGI.

Instancia la clase FastAPI, configura SessionMiddleware necesario para firmar criptográficamente las cookies de sesión (usando secret_key), vincula la ruta /static al directorio físico para servir CSS, registra los APIRouter de auth y services para exponer sus endpoints.

Uso Aplicación



The image shows a login form with the title "Welcome". It contains two input fields: "Username:" with the text "resident1" and "Password:". Below the password field is a green "Sign In" button. Underneath the button is a link "Just browsing?". At the bottom is a grey button labeled "Continue as Guest".

Se puede entrar tanto como un usuario (ya están preregistrados), o como invitado. Entra como invitado te permite ver los servicios que se pueden reservar, pero no podrán reservar nada.

Welcome, Guest Visitor Role: **Guest** [Logout](#)

Available Services

Gym Max Capacity: 5 people	Login to Reserve
Pool Max Capacity: 3 people	Login to Reserve
Sauna Max Capacity: 2 people	Login to Reserve
Tennis Court Max Capacity: 4 people	Login to Reserve
Music Room Max Capacity: 1 people	Login to Reserve

Si entras con una cuenta registrada puedes reservar el servicio el día y la hora.

Book Gym

Max Capacity per slot: 5 people

Select Date:



Select Time:



[Confirm Reservation](#)

[Cancel and go back](#)

Ejecución

Instalar el requirements para que no haya errores, y ejecutar “uvicorn app.main:app -- reload”

Acceso: Navegar a <http://127.0.0.1:8000>

-Usuarios preregistrados:

Resident1 / pass1

Resident2 / pass2